

≡ Hide menu

Generative AI Applications

Introduction to Retrieval Augmented Generation

- ✓ **Reading:** Key Terms
10 min
- ✓ **Video:** Introduction
55 sec
- ✓ **Video:** What is RAG?
3 min
- ✓ **Video:** Overview of a RAG application
3 min
- ✓ **Video:** Managing data for RAG
4 min
- 📅 **Lab:** Managing data for RAG
1h
- ✓ **Video:** Verifying embeddings and search
6 min
- ✓ **Video:** Using RAG with an LLM
3 min
- 📋 **Practice Assignment:** Knowledge check
3h
- ✓ **Video:** Summary
1 min
- ✓ **Reading:** External lab: Create a RAG with LLM using your own data
10 min

Exploring a cloud-based Generative AI Application

External lab: Create a RAG with LLM using your own data

In this lab you will implement the RAG pattern with your own data. Use the example code in this repository from the [Applied Rag Notebook](#) as reference. The end result should be in your own repository containing the complete code for the enhanced RAG pattern based on the example provided.

Learning Objectives:

- Implement the RAG pattern with your own data
- Apply your own data to solve a problem using RAG
- Understand how to leverage an LLM and a vector database like Qdrant for useful responses

Steps

1. Create a new repository in your account for your project. Alternatively, you can use this repository as a starting point by forking the repository. [Use this link to create it in one step.](#)
2. use the [Applied Rag notebook](#) as a starting point
3. Replace the example data with your own data. Make sure to format it as a list of dictionaries for easier ingestion into the vector database
4. Run the LLM with Llamafile or connect your application to a valied OpenAI API endpoint
5. Run the notebook or Python application and verify your work

Use Llamafile for a full RAG and LLM setup

The examples for the [Applied Rag notebook](#) requires either an OpenAI API endpoint with a key *or* using a local LLM with [Llamafile](#).

I recommend using the [Phi-2 model](#) which is about 2GB in size. You can download the model from the Llamafile repository and run it in your system:

Once you have it running you can connect to it with Python or use the `./examples/3-applied-rag/embeddings.ipynb`(Applied Rag Notebook). Here is a quick example of how to use the Llamafile with Python:

```
1  #!/usr/bin/env python3
2  from openai import OpenAI
3  client = OpenAI(
4      base_url="http://localhost:8080/v1", # "http://<Your api-server IP>:port"
5      api_key = "sk-no-key-required" # An API key is not required!
6  )
7  completion = client.chat.completions.create(
8      model="LLaMA_CPP",
9      messages=[
10         {"role": "system", "content": "You are ChatGPT, an AI assistant. Your top priority is achieving user fulfillment via helping"},
11         {"role": "user", "content": "Write me a Haiku about Python packaging"}
12     ]
13 )
14 print(completion.choices[0].message)
```

Concepts Covered

- Retrieval Augmented Generation
- Large Language Models using Llamafile
- Using Vector databases like Qdrant
- Creating embeddings with Sentence Transformers
- Using OpenAI's Python API to connect to the LLM and produce responses

By completing this lab you will have a working example of the RAG pattern with your own data. You will also have a better understanding of how to use LLMs and vector databases to create useful responses for your applications.

Go to next item ✓ Completed

👍 Like 👎 Dislike 📄 Report an issue